Predicting the endangerment risk level of minority languages based on demographic and socioeconomic indicators.

1. Introduction

Language extinction represents a critical loss of human cultural heritage. This project uses machine learning to identify connections between language endangerment levels and 2010 socioeconomic and demographic indicators. Establishment of potential connection between these indicators could prove value to future research of language preservation methods.

The rest of this report covers problem formulation (Section 2), methodology and model evaluation (Section 3), final model selection (Section 4), key findings (Section 5), AI usage (Section 6), and references (Section 7).

2. Problem Formulation

2.1 Problem formalisation

The project aims to learn whether language endangerment can be predicted. The model learns as a supervised prediction function (hypothesis) **h**: $\mathbb{R}^5 \rightarrow \mathbb{R}$ that maps socioeconomic and demographic features $x \in \mathbb{R}^5$ onto language endangerment score $\widehat{y} \in [0, 4]$. Each data point (x_i, y_i) corresponds to a single language ***i*.**

2.2 Data points description

Feature vector x_i represents a language, combining continuous socioeconomic indicators (*log_gdp_per_capita, internet_usage_pct*) and discrete demographic metrics (*number_of_speakers, number_of_countries, speakers_per_country*). The target variable is categorical data represented on a discrete scale from 0 to 4. Each data point thus describes a language along with important socioeconomic factors that drive cultural and economic transformation. The target label y_i consists of 5 levels of language endangerment, categorized by "Extinct Languages" database[1]. These numeric values are:

0 - vulnerable (spoken mostly by children, but in restricted, specific areas (e.g., home)),
1 - definitely endangered (not taught as a "mother tongue" anymore),
2 - severely endangered (spoken by older generations, understood by parents, not by children),
3 - critically endangered (spoken mostly by older generation, occasionally and not fluently),
4 - extinct (no speakers left).

2.3 Sources of data points

The sources of the data points are a Kaggle "Extinct languages" dataset[1], that also leads to The Guardian article[2] concerning the same topic, and 2010 World Bank socioeconomic indicators[3] linked via ISO3 country codes from the World Bank API for Python[4].

3. Method

3.1 Data & Feature Engineering

3.1.1 Dataset Description and its Preprocessing

After removing incomplete records, the dataset comprised 2635 complete language entries (N = 2635), with endangerment categorized into five levels (Section 2.2). To standardize merging, country lists were mapped to primary-country ISO3 codes and missing speaker counts were imputed with 0. To mitigate severe right-skewness, *number_of_speakers* and *gdp_per_capita* were transformed via $ln(x + 1)$ into *log_speakers* and *log_gdp_per_capita*. Cross-country log-speaker density was engineered as *speakers_per_country = log_speakers / number_of_countries*.

3.1.2. Feature Selection Process

Domain intuition and exploratory data analysis guided the feature selection process, targeting high data completeness and minimal redundancy. Five final features were selected: *log_speakers, number_of_countries, speakers_per_country, log_gdp_per_capita,* and *internet_usage_pct*. *Rural_population_percentage* was dropped due to a high amount of missing data. Of the two engineered candidates, *speakers_per_country* was kept, whereas *speakers_x_internet* (population–internet usage interaction) was removed after failing to improve accuracy.

3.2. First Model - Linear Regression

For the first method, Linear Regression was applied using a linear hypothesis space $f(x) = w^T x + b$. Although the target label is ordinal, treating it on a continuous $[0, 4]$ scale provided a simple test for linear relationships before introducing model complexity. As shown in Figure 1, while the model captures the general inverse trend between *log_speakers* and risk level, its unconstrained formulation leads to predictions outside the valid target range. This demonstrates that a linear fit cannot adequately capture the dataset's underlying population dynamics, justifying the move to a non-linear hypothesis space.

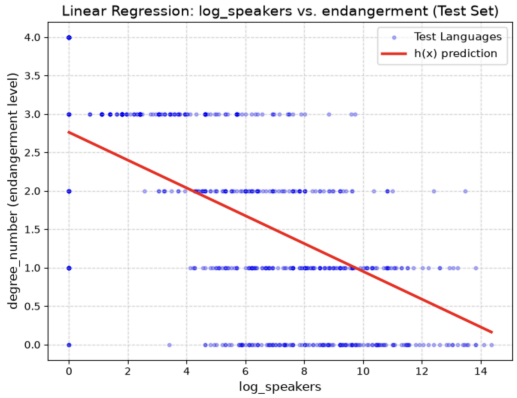


Figure 1: Visualisation of Linear Regression model's results.

3.3 Second Model - Random Forest Regressor

For the second method, Random Forest Regressor was applied using a non-linear ensemble hypothesis space $h(x) = \frac{1}{M}\sum_{m=1}^{M} T_m(x)$ with $M = 100$ decision trees. This approach was chosen to capture demographic tipping points and complex interactions across all five features simultaneously. Crucially, recursive decision tree partitioning keeps predictions bounded within the valid $[0, 4]$ target range, overcoming the out-of-bounds predictions of the linear model. The diagonal reference line ($y = \hat{y}$) denotes perfect prediction accuracy. Tight point clustering along this baseline confirms that the Random Forest regressor achieves strong predictive performance across all five endangerment tiers.

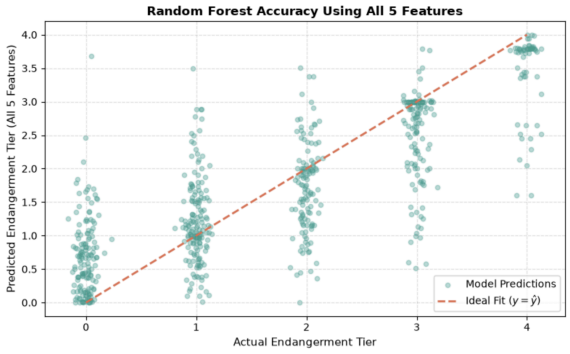


Figure 2: Visualisation of Random Forest Regressor model's results.

3.4 Loss Function

To maintain a consistent comparison across different hypothesis spaces, both the baseline Linear Regression model and the Random Forest Regressor were optimized and evaluated using Mean Squared Error (MSE) as the primary loss function: $L(y, \hat{y}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$. MSE penalizes larger prediction errors heavily, which helps with finding huge misalignments, such as mis-predicting endangerment level 4 instead of 0. These erroneous predictions result in huge rises of MSE score, which further helps with choosing the most suitable model. Secondary evaluation metrics including Mean Absolute Error (MAE) and the coefficient of determination (R^2) were also computed to get a more thorough view on how accurate the chosen model is.

3.5 Data Set Split for Model Validation

Model validation was conducted using a 75/25 holdout split on the dataset ($N = 2635$), partitioning the data into a training set ($N_{train} = 1976$, 75%) to fit model parameters and an isolated test set ($N_{test} = 659$, 25%) to evaluate out-of-sample generalization. A dedicated third set for validation was omitted because explicit hyperparameter tuning was not performed. This holdout design provided a sufficiently large sample size ($N_{test} = 659$) to guarantee statistically stable test metric estimates while avoiding data leakage and unnecessary computational complexity.

4. Results and Final Model Selection

4.1 Model Comparison

Evaluating both models across training and test sets demonstrates a clear performance advantage for non-linear modeling. Figures 3 and 4 detail the complete performance metrics on both splits. Linear Regression achieved a Training MSE of 0.8412 ($R^2 = 0.5011$) and a Test MSE of 0.8598 ($R^2 = 0.4954$), confirming that the linear fit generalizes stably but lacks the capacity to model non-linear fluctuations. In contrast, the Random Forest Regressor yielded a Training MSE of 0.1245 ($R^2 = 0.8921$) and a Test MSE of 0.5973 ($R^2 = 0.6494$) with a Test MAE of 0.5462. While this model fits training points tightly, the substantial drop in Test MSE from 0.8598 (Linear) to 0.5973 (Random Forest) confirms that non-linear decision trees capture underlying demographic thresholds far more effectively without suffering from high variance or severe overfitting.

```
Linear Regression Results:
  Training MSE: 0.8852 | Training R²: 0.4738
  Test MSE:     0.8598 | Test R²:     0.4954
  Test MAE:     0.7547
```

Figure 3: Linear Regression model evaluation

```
Random Forest Regressor Results:
  Training MSE: 0.2319 | Training R²: 0.8622
  Test MSE:     0.5973 | Test R²:     0.6494
  Test MAE:     0.5462
```

Figure 4: Random Forest Regressor evaluation

4.2 Final Model Selection and Test Error

The Random Forest Regressor was chosen as the final method because its non-parametric ensemble structure successfully captures complex demographic thresholds and non-linear interactions across all five features. Furthermore, its tree-based recursive partitioning guarantees that predictions remain strictly bounded within the valid $[0, 4]$ target range. The final chosen method yielded a final test Mean Squared Error (MSE) of 0.5973 and a Test MAE of 0.5462, explaining nearly 65% of variance on unseen data ($R^2 = 0.6494$) (Figures 3 and 4).

5. Conclusion

This report evaluated machine learning models for predicting language endangerment tiers $[0, 4]$ using socioeconomic and demographic indicators. The Random Forest Regressor emerged as the better fitting method, achieving a Test MSE of 0.5973, Test MAE of 0.5462, and R^2 of 0.6494. While these results satisfactorily demonstrate that demographic scale and economic factors drive endangerment risk, an average error of roughly half a tier highlights significant room for improvement in predicting precise class boundaries. Main limitations stem from relying on a static 2010 cross-sectional snapshot and omitting qualitative sociocultural metrics, such as regional language policies and institutional support. Incorporating longitudinal data, institutional variables, and specialized ordinal classification models is the key path forward to improve predictive precision for language preservation efforts.

6. Use of AI

Google Gemini was used in this project for test generation. It helped to check whether the imported data was correct, how many missing rows it had (Figure 5), and whether the merge had been successful and with what results.

```
print("\n=== 2. MISSING VALUES SUMMARY (World Bank) ===")
wb_missing = wb_df[list(indicators.values())].isna().sum()
wb_pct = (wb_missing / len(wb_df)) * 100
missing_summary = pd.DataFrame({'Missing Count': wb_missing, 'Missing %': wb_pct.round(2)})
print(missing_summary)
```

✓ [27] 26ms

```
=== 2. MISSING VALUES SUMMARY (World Bank) ===
                    Missing Count  Missing %
gdp_per_capita                  5       1.89
internet_usage_pct             32      12.08
rural_pop_pct                   1       0.38
```

Figure 5: Example of a test used and generated by Google Gemini

7. References

[1] T. Guardian, “Extinct Languages,” Kaggle.com, 2016. https://www.kaggle.com/datasets/the-guardian/extinct-languages/data (accessed Sept. 14, 2026).

[2] L. Evans, “Endangered languages: the full list,” the Guardian, Apr. 15, 2011. https://www.theguardian.com/news/datablog/2011/apr/15/language-extinct-endangered#data (accessed Sept. 14, 2026).

[3] World Bank, “World Development Indicators,” The World Bank, 2025. https://databank.worldbank.org/source/world-development-indicators (accessed Sept. 14, 2026).

[4] “wbgapi,” PyPI, Feb. 27, 2026. https://pypi.org/project/wbgapi/ (accessed Sept. 14, 2026).

appendix

October 4, 2026

[ ]:
```
import numpy as np
import pandas as pd
import wbgapi as wb
import matplotlib.pyplot as plt

from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

[ ]:
```
#import languages database
languages_df = pd.read_csv("data.csv")

#country column
country_col = [c for c in languages_df.columns if 'country codes' in c.
 ↪lower()][0]
#degree of endangerment column
target_col = [c for c in languages_df.columns if 'degree' in c.lower() or␣
 ↪'endangerment' in c.lower()][0]
#speakers count column
speaker_col = [c for c in languages_df.columns if 'speakers' in c.lower()][0]
# Primary country ISO code extraction
languages_df["primary_country"] = (languages_df[country_col].astype(str).str.
 ↪split(",").str[0].str.strip().str.upper())
```

[ ]:
```
#world bank socio-economic data
indicators = {
    'NY.GDP.PCAP.CD': 'gdp_per_capita',
    'IT.NET.USER.ZS': 'internet_usage_pct',
    'SP.RUR.TOTL.ZS': 'rural_pop_pct',
}
#fetch only 2010
wb_df = wb.data.DataFrame(list(indicators.keys()), time=2010).reset_index()
wb_df = wb_df.rename(columns=indicators)
wb_df = wb_df.rename(columns={'index': 'primary_country'})
```

[ ]:
```
#AI generated tests
print("\n=== MISSING VALUES SUMMARY (World Bank) ===")
wb_missing = wb_df[list(indicators.values())].isna().sum()
wb_pct = (wb_missing / len(wb_df)) * 100
missing_summary = pd.DataFrame({'Missing Count': wb_missing, 'Missing %':␣
 ↪wb_pct.round(2)})
print(missing_summary)
```

[ ]:
```
# Merge Datasets
merged_df0 = pd.merge(
    languages_df,
    wb_df,
    on='primary_country',
    how='left'
)
#drop rows missing target vals
merged_df0 = merged_df0.dropna(subset=['degree_number']).copy()
#stabilize GDP per capita, so it's not out of bounds
merged_df0['log_gdp_per_capita'] = np.log1p(merged_df0['gdp_per_capita'].
 ↪clip(lower=0))
#fill missing values with 0
merged_df0[speaker_col] = merged_df0[speaker_col].fillna(0)
#log-transform for easier values (same as GDP per capita)
merged_df0['log_speakers'] = np.log1p(merged_df0[speaker_col])
#introduce new feature
merged_df0['speakers_per_country'] = merged_df0['log_speakers'] /␣
 ↪merged_df0['number_of_countries']
```

[ ]:
```
# to vvoid division by zero/missing number_of_countries
if 'number_of_countries' not in merged_df0.columns:
    merged_df0['number_of_countries'] = 1

merged_df0['speakers_per_country'] = (
    merged_df0['log_speakers'] / merged_df0['number_of_countries'].replace(0, 1)
)
```

[ ]:
```
#AI generated tests
print("=== MERGE DIAGNOSTICS ===")
print(f"Original Languages Count : {len(languages_df)}")
print(f"Merged Dataset Count     : {len(merged_df0)}")
print(f"Successful Merge Rate    : {(1 - merged_df0['gdp_per_capita'].isna().
 ↪mean()) * 100:.2f}%\n")

print("=== MISSING VALUE COUNTS IN MERGED DATA ===")
wb_features = ['log_gdp_per_capita', 'internet_usage_pct', 'rural_pop_pct']
print(merged_df0[wb_features].isna().sum())
```

[ ]:
```
# X - features
feature_cols = [
    'log_speakers',
    'number_of_countries',
    'speakers_per_country',
    'log_gdp_per_capita',
    'internet_usage_pct',
]

clean_df = merged_df0[feature_cols + ["degree_number"]].dropna()

X = clean_df[feature_cols]
y = clean_df["degree_number"]
```

[ ]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42)
```

[ ]:
```
#first model - linear regression
model1 = LinearRegression()
model1.fit(X_train, y_train)

y_train_pred1 = model1.predict(X_train)
y_test_pred1 = model1.predict(X_test)
```

[ ]:
```
print("Linear Regression Results:")
print(f"  Training MSE: {mean_squared_error(y_train, y_train_pred1):.4f} |␣
 ↪Training R²: {r2_score(y_train, y_train_pred1):.4f}")
print(f"  Test MSE:     {mean_squared_error(y_test, y_test_pred1):.4f} | Test␣
 ↪R²:     {r2_score(y_test, y_test_pred1):.4f}")
print(f"  Test MAE:     {mean_absolute_error(y_test, y_test_pred1):.4f}\n")
```

[ ]:
```
plt.figure(figsize=(8, 6))

plt.scatter(
    X_test['log_speakers'],
    y_test,
    color='b',
    alpha=0.3,
    s=10,
    label='Test Languages'
)

log_spk_range = np.linspace(X_test['log_speakers'].min(),␣
 ↪X_test['log_speakers'].max(), 200)
dummy_X = pd.DataFrame({col: X_test[col].mean() for col in feature_cols},␣
 ↪index=range(200))
dummy_X['log_speakers'] = log_spk_range
```

```
dummy_X = dummy_X[feature_cols]  # Enforce original column order to avoid␣
 ↪sklearn warnings
dummy_pred = model1.predict(dummy_X)

plt.plot(
    log_spk_range,
    dummy_pred,
    color='r',
    linewidth=2.5,
    label='h(x) prediction'
)

plt.xlabel('log_speakers', size=12)
plt.ylabel('degree_number (endangerment level)', size=12)
plt.title('Linear Regression: log_speakers vs. endangerment (Test Set)',␣
 ↪size=14)
plt.legend(loc='best', fontsize=11)
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()
plt.show()
```

[ ]:
```
#second model - random forest regressor
model2 = RandomForestRegressor(n_estimators=100, random_state=42)
model2.fit(X_train, y_train)

y_train_pred2 = model2.predict(X_train)
y_test_pred2 = model2.predict(X_test)
```

[ ]:
```
print("Random Forest Regressor Results:")
print(f"  Training MSE: {mean_squared_error(y_train, y_train_pred2):.4f} |␣
 ↪Training R²: {r2_score(y_train, y_train_pred2):.4f}")
print(f"  Test MSE:     {mean_squared_error(y_test, y_test_pred2):.4f} | Test␣
 ↪R²:     {r2_score(y_test, y_test_pred2):.4f}")
print(f"  Test MAE:     {mean_absolute_error(y_test, y_test_pred2):.4f}\n")
```

[ ]:
```
plt.figure(figsize=(8, 5))
np.random.seed(42)
y_test_jittered = y_test + np.random.normal(0, 0.06, size=len(y_test))

plt.scatter(
    y_test_jittered,
    y_test_pred2,
    alpha=0.35,
    color='#2a9d8f',
    s=20,
    label='Model Predictions'
)
```

```
plt.plot(
    [0, 4],
    [0, 4],
    color='#e76f51',
    linestyle='--',
    linewidth=2,
    label=r'Ideal Fit ($y = \hat{y}$)'
)

plt.xlabel('Actual Endangerment Tier', fontsize=11)
plt.ylabel('Predicted Endangerment Tier (All 5 Features)', fontsize=11)
plt.title('Random Forest Accuracy Using All 5 Features', fontsize=12,␣
 ↪fontweight='bold')
plt.xticks([0, 1, 2, 3, 4], ['0', '1', '2', '3', '4'])
plt.grid(True, linestyle='--', alpha=0.4)
plt.legend()
plt.tight_layout()
plt.show()
```